



NextGen

Efficient Python Lecture 02

CERN
STEAM Academy

07 July 2026

From Arrays to Visual Numerical Experiment

Jacek Gatlik

email: jacek.gatlik@agh.edu.pl

www: <https://galaxy.agh.edu.pl/~jgatlik/>

Faculty of Physics and Applied Computer Science,
AGH University of Krakow, Poland



Outline

Numerical experiment objects

Families of arrays and diagnostics

Matplotlib as a diagnostic instrument

Further reading

Section 1

Numerical experiment objects

From arrays to visual numerical experiments

The first lecture established the language of arrays: axes, shapes, indexing, broadcasting, reductions, and contractions. Here we do not repeat that language as definitions. We use it as a working tool for building small, inspectable numerical experiments.

The guiding idea is simple: a numerical result should be an object we can construct, diagnose, compare, and visualize before it becomes part of a larger code.

We want each array to answer three questions: what physical or numerical quantity does it represent, which axes does it live on, and how can we check that it behaves as expected?

The unit of work: a numerical experiment

A useful numerical experiment is more than a formula evaluated on a computer. It usually contains a domain, a discretization, sampled quantities, transformations, diagnostics, and a visual check.

domain $\longrightarrow$ grid $\longrightarrow$ arrays $\longrightarrow$ diagnostics $\longrightarrow$ plots

For a physicist, this structure is familiar: choose the experimental setup, measure or compute observables, estimate errors, and inspect whether the result is plausible.

A plot is not the last decorative step. It is part of the experiment because it can reveal wrong scales, wrong axes, wrong units, wrong boundary conventions, and wrong assumptions.

A grid is an experimental choice

Choosing a grid fixes what the computer is allowed to resolve. The domain length, number of points, spacing, and endpoint convention are numerical assumptions, not neutral implementation details.

```
1 import numpy as np
2
3 xmin, xmax = -10.0, 10.0
4 nx = 1001
5
6 x = np.linspace(xmin, xmax, nx)
7 dx = x[1] - x[0]
8
9 print(x[0], x[-1], dx)
```

Changing `nx` changes the scale of numerical differentiation and integration. Changing the interval may change boundary effects. A grid is therefore part of the model specification.

Point grids and cell grids

For sampled functions, we often store values at grid points. For finite-volume data, histograms, or image-like fields, it is often better to distinguish cell centres from cell edges.

```
1 xmin, xmax = -5.0, 5.0
2 nx = 400
3
4 x_edges = np.linspace(xmin, xmax, nx + 1)
5 x = 0.5 * (x_edges[:-1] + x_edges[1:])
6
7 dx = np.diff(x_edges)
```

This distinction becomes important later for `pcolormesh`, histograms, conservative discretizations, and any calculation where cell sizes are not only visual decoration.

Sampled quantities are data, not symbolic functions

After sampling, a mathematical expression becomes an array. The array does not know the continuous formula, the units, or the grid unless we keep that information explicitly in the workflow.

```
1 x = np.linspace(-8.0, 8.0, 2000)
2
3 sigma = 1.2
4 k0 = 8.0
5 psi = np.exp(-(x/sigma)**2) * np.exp(1j*k0*x)
6
7 rho = np.abs(psi)**2
```

The arrays `psi` and `rho` are meaningful only together with the coordinate array `x` and the interpretation of the axis.

Keep coordinates and values together

A frequent source of confusion is separating a field from the coordinate grid on which it is defined. Even in a short notebook, it is useful to keep the numerical object and its coordinate information close to one another.

```
1 packet = {  
2     "x": x,  
3     "psi": psi,  
4     "rho": rho,  
5     "dx": x[1] - x[0],  
6     "description": "Gaussian wave packet"  
7 }
```

This is not yet software architecture. It is simply a habit: do not let arrays become anonymous blocks of numbers.

Array contracts before expensive work

Before running an expensive computation, check the assumptions that must be true. These checks are small, but they often catch the scientific bugs that are hardest to see later.

```
1 assert x.ndim == 1
2 assert psi.shape == x.shape
3 assert np.all(np.diff(x) > 0.0)
4 assert np.isfinite(rho).all()
5 assert np.all(rho >= 0.0)
```

These are not only programming assertions. They express numerical contracts: the grid is ordered, the field lives on that grid, and the density is finite and non-negative.

Diagnostics before visualization

Before plotting a field, compute a few numbers that summarize what we expect to see. For a localized density, natural diagnostics are norm, centre, width, and peak position.

```
1 norm = np.trapezoid(rho, x)
2
3 x_mean = np.trapezoid(x * rho, x) / norm
4 width = np.sqrt(np.trapezoid((x - x_mean)**2 * rho, x) / norm)
5
6 imax = np.argmax(rho)
7 x_peak = x[imax]
```

If the plot later shows a peak far away from `x_peak`, the problem may be in the plotting code, axis scaling, or data selection rather than in the physics.

Discrete derivatives depend on boundary meaning

The same array can represent a function on an interval or a periodic function on a circle. The numerical derivative must know which interpretation is intended.

```
1 u = np.sin(2*np.pi*x / (x[-1] - x[0]))
2 dx = x[1] - x[0]
3
4 # Interior derivative on a finite interval
5 ux_inner = (u[2:] - u[:-2]) / (2*dx)
6
7 # Periodic derivative on a periodic grid
8 ux_periodic = (np.roll(u, -1) - np.roll(u, 1)) / (2*dx)
```

The two arrays are not interchangeable. Boundary conditions are part of the numerical experiment.

Masks as physical subdomains

A Boolean mask can represent a physical region, an analysis window, a detector acceptance range, or a part of the domain where a diagnostic is meaningful.

```
1 window = np.abs(x - x_mean) < 2.0 * width
2
3 local_norm = np.trapezoid(rho[window], x[window])
4 local_fraction = local_norm / norm
```

The mask should be interpreted geometrically: it is a discrete version of a condition such as $|x - \langle x \rangle| < 2\sigma$.

A diagnostic packet

For a small experiment, a compact diagnostic object is often more useful than several unrelated print statements. It can be displayed, logged, compared, or attached to a figure.

```
1 diagnostics = {  
2     "norm": norm,  
3     "x_mean": x_mean,  
4     "width": width,  
5     "x_peak": x_peak,  
6     "local_fraction": local_fraction,  
7 }  
8  
9 for name, value in diagnostics.items():  
10     print(f"{name:>14s} = {value:.6g}")
```

The goal is not to introduce a complicated framework. The goal is to make numerical checks visible and repeatable.

Section 2

Families of arrays and diagnostics

Many experiments, one structured array

A scientific calculation rarely produces only one curve. More often, we repeat the same construction for several parameter values, initial conditions, resolutions, or noise levels. Instead of creating variables such as `u1`, `u2`, and `u3`, we add an experiment axis.

$$U_{pi} = U(p_p, x_i).$$

The first axis labels the experiment. The second axis labels position. This makes comparison and visualization systematic.

A parameter axis is a design decision

Here a whole family of damped oscillatory signals is stored in one array. The shape tells us how to read the result.

```
1 x_sweep = np.linspace(0.0, 12.0, 800)
2 gamma = np.array([0.05, 0.10, 0.20, 0.40])
3
4 U = np.exp(-gamma[:, None] * x_sweep[None, :]) * np.sin(4.0*x_sweep[None, :])
5
6 print(U.shape)      # (number of gamma values, number of grid points)
```

The added axis is not a trick. It is a computational representation of a parameter sweep.

Normalize along the physical axis

When each row is one experiment, normalization must be performed along the grid axis while preserving the parameter axis.

```
1 amplitude = np.max(np.abs(U), axis=1, keepdims=True)
2 U_scaled = U / amplitude
3
4 print(amplitude.shape)
5 print(U_scaled.shape)
```

The option `keepdims=True` is not cosmetic. It keeps the result broadcast-compatible with the original array and makes the intended axis structure explicit.

Diagnostics per experiment

Once the axes are clear, diagnostics can be computed for the whole family without changing the structure of the workflow.

```
1 energy_like = np.trapezoid(U**2, x_sweep, axis=1)
2 max_abs = np.max(np.abs(U), axis=1)
3 final_value = U[:, -1]
4
5 summary = np.column_stack([gamma, energy_like, max_abs, final_value])
```

The array `summary` is small, but it still has a numerical interpretation: each row corresponds to one parameter value and each column to one diagnostic.

Locating features in a sweep

The location of a maximum, zero crossing, or front position is often more informative than the full field. Feature extraction can also be vectorized over the experiment axis.

```
1 idx_peak = np.argmax(np.abs(U), axis=1)
2 x_peak = x_sweep[idx_peak]
3
4 peak_value = U[np.arange(U.shape[0]), idx_peak]
```

The indexing line says: for each experiment row, take the value at its own peak location. This is a common pattern when converting fields into observables.

Comparing against a reference

A reference solution may come from an analytical formula, a high-resolution run, a conservation law, or a limiting approximation. The comparison should preserve local information before reducing it.

```
1 U_ref = np.sin(4.0*x_sweep)[None, :]      # undamped reference
2
3 E = U - U_ref                             # local error array
4 err_inf = np.max(np.abs(E), axis=1)
5 err_rms = np.sqrt(np.mean(E**2, axis=1))
```

A single error norm is useful, but it hides where the approximation fails. The error array is therefore also a plotting object.

Residuals as arrays

In differential-equation problems, the residual tells us how well a sampled object satisfies an equation. It is not only a machine-learning concept; it is a classical numerical diagnostic. For $u(x) = \sin(kx)$, the equation $u_{xx} + k^2u = 0$ should be satisfied up to discretization error.

```
1 k = 3.0
2 x = np.linspace(0.0, np.pi, 400)
3 u = np.sin(k*x)
4 dx = x[1] - x[0]
5
6 uxx = (u[2:] - 2*u[1:-1] + u[:-2]) / dx**2
7 residual = uxx + k**2 * u[1:-1]
8
9 res_norm = np.max(np.abs(residual))
```

Convergence data as an output

A convergence test is a parameter sweep where the parameter is the grid spacing. The output should be stored as data that can be inspected and plotted.

```
1 def residual_norm(nx, k=3.0):
2     x = np.linspace(0.0, np.pi, nx)
3     u = np.sin(k*x)
4     dx = x[1] - x[0]
5     uxx = (u[2:] - 2*u[1:-1] + u[:-2]) / dx**2
6     r = uxx + k**2 * u[1:-1]
7     return dx, np.max(np.abs(r))
8
9 sizes = np.array([50, 100, 200, 400, 800])
10 data = np.array([residual_norm(n) for n in sizes])
11
12 dx_values = data[:, 0]
13 errors = data[:, 1]
```

Estimating an observed order

A convergence plot is more convincing when accompanied by a numerical estimate of the observed order. If $e \sim C\Delta x^p$, then ratios of neighbouring errors estimate p .

```
1 orders = np.log(errors[:-1] / errors[1:]) \
2         / np.log(dx_values[:-1] / dx_values[1:])
3
4 for n1, n2, p in zip(sizes[:-1], sizes[1:], orders):
5     print(f"{n1:4d} -> {n2:4d}: observed order {p:.2f}")
```

This prepares the visual check: on a log-log plot, a second-order method should produce a line with slope close to two.

Randomness as part of the experiment

Synthetic data are useful for testing analysis and plotting code because we know the signal, the noise model, and the expected residual scale.

```
1 rng = np.random.default_rng(seed=123)
2
3 t = np.linspace(0.0, 10.0, 300)
4 clean = np.exp(-0.25*t) * np.sin(3*t)
5 sigma_noise = 0.08
6
7 measured = clean + sigma_noise * rng.normal(size=t.size)
8 residual = measured - clean
9
10 rms_residual = np.sqrt(np.mean(residual**2))
```

The random generator is explicit, so the experiment can be repeated exactly.

Invalid values are numerical evidence

Non-finite values should not be silently ignored. They often indicate overflow, division by zero, invalid input ranges, or a model being evaluated outside its domain of validity.

```
1 y = np.log(measured - measured.min())
2
3 bad = ~np.isfinite(y)
4 print("number of invalid values:", bad.sum())
5
6 valid_mean = np.mean(y[~bad])
```

A mask of invalid values is itself diagnostic data. Later, plotting where invalid values occur can reveal a modelling or preprocessing mistake.

A compact one-dimensional experiment

The elements above can be wrapped into a small, inspectable function. It constructs the grid, samples a field, computes diagnostics, and returns objects ready for plotting.

```
1 def wave_packet_experiment(nx=2000, sigma=1.2, k0=8.0):
2     x = np.linspace(-8.0, 8.0, nx)
3     psi = np.exp(-(x/sigma)**2) * np.exp(1j*k0*x)
4     rho = np.abs(psi)**2
5
6     norm = np.trapezoid(rho, x)
7     x_mean = np.trapezoid(x*rho, x) / norm
8     width = np.sqrt(np.trapezoid((x - x_mean)**2*rho, x) / norm)
9
10    diagnostics = {"norm": norm, "x_mean": x_mean, "width": width}
11    return x, rho, diagnostics
12
13 x, rho, diagnostics = wave_packet_experiment()
```

Section 3

Matplotlib as a diagnostic instrument

Plotting as numerical validation

A plot should help us answer a numerical question. Does the solution have the expected scale? Is the peak in the expected place? Are the boundary values plausible? Does the residual have structure? Does the convergence rate match the method?

Every figure in a numerical notebook should be able to justify its existence: it should check a scale, a structure, a comparison, or a failure mode.

This is why plotting belongs inside the numerical workflow, not only at the end of a project.

Use the object-oriented Matplotlib interface

For scientific work, it is best to create the figure and axes explicitly. This makes the plotting code easier to modify, combine, save, and reuse.

```
1 import matplotlib.pyplot as plt
2
3 fig, ax = plt.subplots(figsize=(6.0, 3.2))
4
5 ax.plot(x, rho)
6 ax.set_xlabel(r"$x$")
7 ax.set_ylabel(r"$|\psi(x)|^2$")
8 ax.set_title("Gaussian wave packet density")
```

The variable `ax` is the object on which we place curves, labels, legends, scales, annotations, and limits.

First diagnostic plot: data and expectation

A useful first plot compares the computed object with a simple expectation. For the wave packet density, we can mark the measured centre and width.

```
1 fig, ax = plt.subplots(figsize=(6.0, 3.2))
2
3 ax.plot(x, rho, label=r"$|\psi|^2$")
4 ax.axvline(diagnostics["x_mean"], linestyle="--", label="centre")
5
6 left = diagnostics["x_mean"] - diagnostics["width"]
7 right = diagnostics["x_mean"] + diagnostics["width"]
8 ax.axvspan(left, right, alpha=0.2, label="one width")
9
10 ax.set_xlabel(r"$x$")
11 ax.set_ylabel(r"density")
12 ax.legend()
```

Labels, units, and ranges are part of the check

A plot without units and axis ranges is hard to interpret scientifically. Labels should say what the plotted quantities mean, not only what variable names were used in code.

```
1 fig, ax = plt.subplots(figsize=(6.0, 3.2))
2
3 ax.plot(t, measured, ".", markersize=3, label="synthetic measurement")
4 ax.plot(t, clean, linewidth=2, label="known signal")
5
6 ax.set_xlabel(r"time $t$ [arb. units]")
7 ax.set_ylabel(r"signal $s(t)$")
8 ax.set_xlim(0.0, 10.0)
9 ax.grid(True, alpha=0.3)
10 ax.legend()
```

Residuals deserve their own plot

When data and model are plotted on the same axes, small systematic discrepancies may be hidden. A residual plot makes the error structure visible.

```
1 fig, ax = plt.subplots(figsize=(6.0, 2.8))
2
3 ax.axhline(0.0, linewidth=1)
4 ax.plot(t, residual, ".", markersize=3)
5
6 ax.set_xlabel(r"time $t$ [arb. units]")
7 ax.set_ylabel(r"residual")
8 ax.set_title(f"RMS residual = {rms_residual:.3f}")
9 ax.grid(True, alpha=0.3)
```

A residual with visible structure is often more important than a small global error norm.

Plotting a parameter sweep

When plotting a sweep, the visual encoding should reflect the array structure. Each row of U corresponds to one parameter value.

```
1 fig, ax = plt.subplots(figsize=(6.0, 3.2))
2
3 for g, row in zip(gamma, U):
4     ax.plot(x_sweep, row, label=fr"$\gamma={g:.2f}$")
5
6 ax.set_xlabel(r"$x$")
7 ax.set_ylabel(r"$u(x; \gamma)$")
8 ax.legend(title="damping")
```

For a small number of curves, direct labelling is readable. For many curves, a colour map or image representation may be better.

Convergence plot on logarithmic axes

Convergence is naturally checked on logarithmic axes. A reference slope turns the plot from a picture into a diagnostic test.

```
1 fig, ax = plt.subplots(figsize=(5.5, 3.2))
2
3 ax.loglog(dx_values, errors, "o-", label="measured error")
4 ax.loglog(dx_values,
5           errors[-1] * (dx_values/dx_values[-1])**2,
6           "--", label=r"$0(\Delta x^2)$")
7
8 ax.set_xlabel(r"grid spacing $\Delta x$")
9 ax.set_ylabel(r"$\|r\|_{\infty}$")
10 ax.invert_xaxis()
11 ax.grid(True, which="both", alpha=0.3)
12 ax.legend()
```

Subplots for controlled comparison

Subplots are useful when two quantities should be compared without forcing them onto the same vertical scale. A common diagnostic layout is: data and model above, residual below.

```
1 fig, axes = plt.subplots(2, 1, sharex=True, figsize=(6.0, 4.0))
2
3 axes[0].plot(t, measured, ".", markersize=3, label="data")
4 axes[0].plot(t, clean, linewidth=2, label="model")
5 axes[0].set_ylabel(r"signal  $s(t)$ ")
6 axes[0].legend()
7
8 axes[1].axhline(0.0, linewidth=1)
9 axes[1].plot(t, residual, ".", markersize=3)
10 axes[1].set_xlabel(r"time  $t$ ")
11 axes[1].set_ylabel("residual")
12
13 fig.tight_layout()
```

The shared horizontal axis makes the comparison local: a feature in the residual can be connected immediately with the corresponding part of the signal.

→ **Example 02 - 01**

Shared axes and fair comparison

When several panels show related quantities, axis limits should be chosen deliberately. Otherwise a visual comparison can become misleading.

```
1 fig, axes = plt.subplots(1, 2, sharex=True, sharey=True,
2                           figsize=(7.0, 3.0))
3
4 axes[0].plot(x_sweep, U[0])
5 axes[0].set_title(fr"$\gamma={gamma[0]:.2f}$")
6
7 axes[1].plot(x_sweep, U[-1])
8 axes[1].set_title(fr"$\gamma={gamma[-1]:.2f}$")
9
10 for ax in axes:
11     ax.set_xlabel(r"$x$")
12     ax.grid(True, alpha=0.3)
13
14 axes[0].set_ylabel(r"$u(x)$")
15 fig.tight_layout()
```

Shared axes are often a simple way to prevent the plot from exaggerating or hiding differences.

Uncertainty bands

When an observable has an estimated uncertainty, the plot should show it. For continuous curves, an uncertainty band is often easier to read than many error bars.

```
1 mean = clean
2 std = sigma_noise * np.ones_like(t)
3
4 fig, ax = plt.subplots(figsize=(6.0, 3.2))
5
6 ax.plot(t, measured, ".", markersize=3, label="data")
7 ax.plot(t, mean, linewidth=2, label="model")
8 ax.fill_between(t, mean - std, mean + std, alpha=0.25,
9                label=r"$\pm 1\sigma$")
10
11 ax.set_xlabel(r"time $t$")
12 ax.set_ylabel(r"signal $s(t)$")
13 ax.legend()
```

The band changes the interpretation of the residual: not every nonzero difference is physically or statistically meaningful.

Residual distributions

A residual time series shows where the mismatch occurs. A residual distribution shows whether the scale and symmetry of the mismatch are plausible.

```
1 fig, ax = plt.subplots(figsize=(5.5, 3.2))
2
3 ax.hist(residual, bins=35, density=True, alpha=0.7)
4 ax.axvline(0.0, linewidth=1)
5
6 ax.set_xlabel("residual")
7 ax.set_ylabel("density")
8 ax.set_title(f"mean = {residual.mean():.3g}, std = {residual.std():.3g}")
```

This plot discards temporal ordering, but it exposes bias, heavy tails, outliers, and non-Gaussian structure.

Dense scatter data

Scatter plots are useful for paired observations, phase-space views, and comparing predicted and measured quantities. With many points, small markers or transparency can prevent overplotting.

```
1 rng = np.random.default_rng(123)
2
3 x_data = rng.normal(size=5000)
4 y_data = 0.7*x_data + 0.4*rng.normal(size=x_data.size)
5
6 fig, ax = plt.subplots(figsize=(4.5, 4.0))
7 ax.scatter(x_data, y_data, s=6, alpha=0.25)
8
9 ax.set_xlabel(r"$x$")
10 ax.set_ylabel(r"$y$")
11 ax.set_aspect("equal", adjustable="box")
```

The visual density of points is already information. It should not be hidden by large markers.

From curves to fields

Many physical objects are scalar fields rather than one-dimensional curves: temperature maps, density fields, residual maps, wave amplitudes, potentials, and detector images. A sampled two-dimensional field has the structure

$$F_{ij} = f(x_i, y_j).$$

The indices are not only array positions. They represent two coordinate directions. A reliable visualization must preserve this coordinate meaning.

When we plot a two-dimensional array, are we looking at array indices, pixel coordinates, or physical coordinates?

Constructing a two-dimensional grid

The choice of indexing convention should be explicit. With `indexing="ij"`, the first axis corresponds to x and the second axis corresponds to y .

```
1 x2 = np.linspace(-5.0, 5.0, 300)
2 y2 = np.linspace(-3.0, 3.0, 180)
3
4 X2, Y2 = np.meshgrid(x2, y2, indexing="ij")
5
6 print(X2.shape, Y2.shape)      # (300, 180), (300, 180)
7 print(X2[10, 20], Y2[10, 20])
```

This convention is close to the mathematical notation $f(x_i, y_j)$, but image display functions may use row-column conventions that require care.

Sampling a scalar field

Once the coordinate arrays exist, formulas can be written almost exactly as mathematical expressions. Broadcasting happens through the grid arrays.

```
1 sigma_x = 1.1
2 sigma_y = 0.7
3 x0, y0 = 1.0, -0.4
4
5 rho2 = np.exp(-((X2 - x0)**2/(2*sigma_x**2)
6               + (Y2 - y0)**2/(2*sigma_y**2)))
7
8 R2 = np.sqrt(X2**2 + Y2**2)
9 wave2 = np.sin(3*R2) * np.exp(-0.15*R2**2)
```

The arrays `rho2` and `wave2` have the same grid but represent different fields. This makes comparisons and overlays straightforward.

Two-dimensional diagnostics

Before making a field plot, compute simple diagnostics. For a localized non-negative field, natural quantities are integral, centre of mass, and maximum location.

```
1 dx2 = x2[1] - x2[0]
2 dy2 = y2[1] - y2[0]
3
4 mass2 = np.sum(rho2) * dx2 * dy2
5 x_cm = np.sum(X2 * rho2) * dx2 * dy2 / mass2
6 y_cm = np.sum(Y2 * rho2) * dx2 * dy2 / mass2
7
8 imax, jmax = np.unravel_index(np.argmax(rho2), rho2.shape)
9 x_peak, y_peak = x2[imax], y2[jmax]
```

These numbers should agree with the geometry seen in the figure. If they do not, inspect the coordinate convention first.

First image: array coordinates

`imshow` can display a two-dimensional array immediately. By default, however, it shows array coordinates rather than the physical x - and y -coordinates.

```
1 fig, ax = plt.subplots(figsize=(5.0, 3.5))
2
3 im = ax.imshow(rho2)
4 fig.colorbar(im, ax=ax, label="array value")
5
6 ax.set_xlabel("column index")
7 ax.set_ylabel("row index")
8 ax.set_title("Displayed in array coordinates")
```

This is useful for debugging raw arrays, but it is usually not enough for a scientific field plot.

Physical axes with `imshow`

To interpret the image as a field on a physical domain, specify the extent and the origin. With the `indexing="ij"` convention used above, transposition is needed for the displayed image.

```
1 fig, ax = plt.subplots(figsize=(5.5, 3.5))
2
3 im = ax.imshow(
4     rho2.T,
5     extent=[x2.min(), x2.max(), y2.min(), y2.max()],
6     origin="lower",
7     aspect="auto"
8 )
9
10 ax.set_xlabel(r"$x$")
11 ax.set_ylabel(r"$y$")
12 fig.colorbar(im, ax=ax, label=r"$\rho(x,y)$")
```

The transpose is not arbitrary; it reconciles mathematical axis order with image row-column order.

A quick convention check

When an image looks rotated or reflected, do not guess. Place an asymmetric feature in a known position and check whether it appears where it should.

```
1 print("expected peak:", x_peak, y_peak)
2
3 fig, ax = plt.subplots(figsize=(5.5, 3.5))
4 im = ax.imshow(rho2.T, extent=[x2.min(), x2.max(), y2.min(), y2.max()],
5                   origin="lower", aspect="auto")
6
7 ax.plot(x_peak, y_peak, "o", label="maximum")
8 ax.plot(x_cm, y_cm, "x", markersize=8, label="centre of mass")
9
10 ax.set_xlabel(r"$x$")
11 ax.set_ylabel(r"$y$")
12 ax.legend()
13 fig.colorbar(im, ax=ax, label=r"$\rho$")
```

pcolormesh keeps coordinates explicit

For fields defined on coordinate grids, `pcolormesh` is often more transparent than `imshow`. The coordinate arrays appear directly in the plotting command.

```
1 fig, ax = plt.subplots(figsize=(5.5, 3.5))
2
3 pcm = ax.pcolormesh(X2, Y2, rho2, shading="auto")
4
5 ax.set_xlabel(r"$x$")
6 ax.set_ylabel(r"$y$")
7 ax.set_aspect("equal")
8 fig.colorbar(pcm, ax=ax, label=r"$\rho(x,y)$")
```

This is especially useful when the grid is nonuniform, transformed, or generated from another coordinate system.

→ **Example 02 - 02**

Nonuniform coordinate grids

A nonuniform grid may be useful when more resolution is needed near a localized structure. The visualization should use the actual coordinates, not assume equal spacing.

```
1 s = np.linspace(-1.0, 1.0, 250)
2 x_nonuniform = 5.0 * s**3
3
4 y_nonuniform = np.linspace(-3.0, 3.0, 180)
5 Xn, Yn = np.meshgrid(x_nonuniform, y_nonuniform, indexing="ij")
6
7 rho_nonuniform = np.exp(-(Xn**2 + Yn**2))
8
9 fig, ax = plt.subplots(figsize=(5.5, 3.5))
10 pcm = ax.pcolormesh(Xn, Yn, rho_nonuniform, shading="auto")
11 ax.set_aspect("equal")
12 fig.colorbar(pcm, ax=ax, label=r"$\rho$")
```

Here the coordinate arrays are not decoration. They define the geometry of the plotted cells.

Colorbars carry units and scale

A colorbar is a numerical axis. It should be labelled with the quantity and, when applicable, the physical units.

```
1 fig, ax = plt.subplots(figsize=(5.5, 3.5))
2
3 pcm = ax.pcolormesh(X2, Y2, rho2, shading="auto")
4 cbar = fig.colorbar(pcm, ax=ax)
5 cbar.set_label(r"density $\rho$ [arb. units]")
6
7 ax.set_xlabel(r"$x$ [arb. units]")
8 ax.set_ylabel(r"$y$ [arb. units]")
9 ax.set_aspect("equal")
```

Without the colorbar, a field plot may show structure but not magnitude.

Sequential, diverging, and cyclic quantities

The colormap should match the mathematical meaning of the field. Colour is not only style; it defines how numerical values are visually ordered.

- ▶ Sequential: density, probability, energy, intensity, magnitude.
- ▶ Diverging: signed residual, difference, deviation from a reference.
- ▶ Cyclic: phase, angle, orientation modulo 2π .

```
1 fig, ax = plt.subplots(figsize=(5.5, 3.5))
2 pcm = ax.pcolormesh(X2, Y2, wave2, shading="auto", cmap="RdBu_r")
3 fig.colorbar(pcm, ax=ax, label=r"signed field $w(x,y)$")
```

The important point is the match between colour ordering and numerical interpretation.

Centred colour scale for residuals

For residuals or signed errors, zero should be visually meaningful. A symmetric colour range makes positive and negative deviations comparable.

```
1 rho_approx = rho2 * (1.0 + 0.15*np.sin(4.0*X2))
2 res2 = rho_approx - rho2
3
4 limit = np.max(np.abs(res2))
5
6 fig, ax = plt.subplots(figsize=(5.5, 3.5))
7 pcm = ax.pcolormesh(X2, Y2, res2, shading="auto",
8                     cmap="RdBu_r", vmin=-limit, vmax=limit)
9
10 ax.set_xlabel(r"$x$")
11 ax.set_ylabel(r"$y$")
12 fig.colorbar(pcm, ax=ax, label="signed residual")
```

Controlling false contrast

Automatic colour limits can make small numerical noise look dramatic or hide important structure behind a few outliers. Colour limits should be chosen and reported consciously.

```
1 noisy_field = rho2.copy()
2 noisy_field[20, 20] = 20.0 * noisy_field.max()    # one outlier
3
4 vmin, vmax = np.percentile(noisy_field, [1, 99])
5
6 fig, ax = plt.subplots(figsize=(5.5, 3.5))
7 pcm = ax.pcolormesh(X2, Y2, noisy_field, shading="auto",
8                     vmin=vmin, vmax=vmax)
9
10 ax.set_title("colour range: 1st--99th percentile")
11 fig.colorbar(pcm, ax=ax, label=r"$\rho$")
```

Clipping can be useful, but it must be explicit because it changes what the viewer can see.

Aspect ratio and geometry

A numerical field may look distorted if the aspect ratio of the axes is not controlled. This is important whenever distances, angles, circular structures, or wave fronts have physical meaning.

```
1 fig, axes = plt.subplots(1, 2, figsize=(8.0, 3.2), sharex=True, sharey=True)
2
3 for ax in axes:
4     ax.pcolormesh(X2, Y2, rho2, shading="auto")
5     ax.set_xlabel(r"$x$")
6     ax.set_ylabel(r"$y$")
7
8 axes[0].set_title("default aspect")
9 axes[1].set_title("equal aspect")
10 axes[1].set_aspect("equal")
11
12 fig.tight_layout()
```

If geometry matters, `set_aspect("equal")` is often a diagnostic necessity, not an aesthetic preference.

Contours and level sets

Contours are useful when level sets have physical meaning: wave fronts, equipotential lines, phase boundaries, threshold regions, or zero residual curves.

```
1 fig, ax = plt.subplots(figsize=(5.5, 3.5))
2
3 pcm = ax.pcolormesh(X2, Y2, wave2, shading="auto", alpha=0.8)
4 cs = ax.contour(X2, Y2, wave2, levels=8, linewidths=0.8)
5 ax.clabel(cs, fontsize=8)
6
7 ax.set_xlabel(r"$x$")
8 ax.set_ylabel(r"$y$")
9 fig.colorbar(pcm, ax=ax, label=r"$w(x,y)$")
```

A contour plot can reveal geometry that is less obvious in colour alone.

Masks and thresholds on fields

A Boolean mask can mark a physical region, an invalid region, or a threshold-defined structure. Plotting the mask together with the field makes the analysis choice visible.

```
1 mask = rho2 > 0.5 * rho2.max()
2 masked_wave = np.ma.array(wave2, mask=~mask)
3
4 fig, ax = plt.subplots(figsize=(5.5, 3.5))
5 pcm = ax.pcolormesh(X2, Y2, masked_wave, shading="auto")
6 ax.contour(X2, Y2, mask, levels=[0.5], linewidths=1.5)
7
8 ax.set_xlabel(r"$x$")
9 ax.set_ylabel(r"$y$")
10 ax.set_aspect("equal")
11 fig.colorbar(pcm, ax=ax, label=r"$w$ inside selected region")
```

This is safer than silently deleting data or plotting only an unexplained subset.

Residual maps for differential equations

For a sampled field, a residual can be another field. For example, if $u(x, y) = \sin(kx) \sin(l y)$, then

$$u_{xx} + u_{yy} + (k^2 + l^2)u = 0.$$

A finite-difference residual checks the sampled equation locally.

```
1 k, ell = 2.0, 3.0
2 u2 = np.sin(k*X2) * np.sin(ell*Y2)
3
4 uxx = (u2[2:, 1:-1] - 2*u2[1:-1, 1:-1] + u2[:-2, 1:-1]) / dx2**2
5 uyy = (u2[1:-1, 2:] - 2*u2[1:-1, 1:-1] + u2[1:-1, :-2]) / dy2**2
6
7 R_pde = uxx + uyy + (k**2 + ell**2) * u2[1:-1, 1:-1]
```

The residual has shape smaller than the original field because this stencil used only interior points.

Plotting a residual map

A residual norm gives one number. A residual map shows where the numerical equation is least satisfied.

```
1 Xi = X2[1:-1, 1:-1]
2 Yi = Y2[1:-1, 1:-1]
3
4 fig, ax = plt.subplots(figsize=(5.5, 3.5))
5
6 lim = np.max(np.abs(R_pde))
7 pcm = ax.pcolormesh(Xi, Yi, R_pde, shading="auto",
8                     cmap="RdBu_r", vmin=-lim, vmax=lim)
9
10 ax.set_xlabel(r"$x$")
11 ax.set_ylabel(r"$y$")
12 ax.set_title(fr"$\|R\|_{\infty} = \{lim:.2e\}$")
13 fig.colorbar(pcm, ax=ax, label="residual")
```

This plot can reveal boundary artefacts, anisotropy, local singular behaviour, or indexing mistakes.

→ **Example 02 - 03**

Time evolution as an image

A time-dependent one-dimensional field $u(t, x)$ can be displayed as a two-dimensional image. One axis represents time, one axis represents space, and colour represents field value.

```
1 x_time = np.linspace(-10.0, 10.0, 400)
2 t_time = np.linspace(0.0, 8.0, 300)
3
4 Tg, Xg = np.meshgrid(t_time, x_time, indexing="ij")
5 front = -6.0 + 1.2*Tg
6 Utx = np.tanh((Xg - front) / 0.5)
7
8 fig, ax = plt.subplots(figsize=(6.0, 3.5))
9 im = ax.imshow(Utx.T, extent=[t_time.min(), t_time.max(),
10                               x_time.min(), x_time.max()],
11                origin="lower", aspect="auto", cmap="RdBu_r")
12
13 ax.set_xlabel("time")
14 ax.set_ylabel(r"$x$")
15 fig.colorbar(im, ax=ax, label=r"$u(t, x)$")
```

Parameter sweep as an image

A family of one-dimensional curves can also be visualized as an image. This is helpful when there are too many curves for a readable line plot.

```
1 x_scan = np.linspace(0.0, 12.0, 600)
2 gamma_scan = np.linspace(0.02, 0.5, 120)
3
4 U_scan = np.exp(-gamma_scan[:, None] * x_scan[None, :]) \
5           * np.sin(4*x_scan[None, :])
6
7 fig, ax = plt.subplots(figsize=(6.0, 3.5))
8 im = ax.imshow(U_scan, extent=[x_scan.min(), x_scan.max(),
9                                gamma_scan.min(), gamma_scan.max()],
10               origin="lower", aspect="auto", cmap="RdBu_r")
11
12 ax.set_xlabel(r"$x$")
13 ax.set_ylabel(r"$\gamma$")
14 fig.colorbar(im, ax=ax, label=r"$u(x; \gamma)$")
```

The parameter axis becomes a visual dimension of the experiment.

Tracking a feature

Images show the full field, but extracted features often reveal the dynamics more clearly. For a travelling front, one may track the zero level set or another characteristic position.

```
1 front_position = []
2
3 for row in Utx:
4     j = np.argmin(np.abs(row))
5     front_position.append(x_time[j])
6
7 front_position = np.array(front_position)
8
9 fig, ax = plt.subplots(figsize=(6.0, 3.2))
10 ax.plot(t_time, front_position)
11 ax.set_xlabel("time")
12 ax.set_ylabel("feature position")
```

Feature tracking converts a field into a lower-dimensional observable. The image is still useful for checking whether the extracted feature is meaningful.

Overlaying a feature on a space-time plot

A powerful sanity check is to overlay an extracted feature on the field from which it was computed.

```
1 fig, ax = plt.subplots(figsize=(6.0, 3.5))
2
3 im = ax.imshow(Utx.T, extent=[t_time.min(), t_time.max(),
4                               x_time.min(), x_time.max()],
5                 origin="lower", aspect="auto", cmap="RdBu_r")
6
7 ax.plot(t_time, front_position, linestyle="--", linewidth=1.5,
8         label="tracked feature")
9 ax.set_xlabel("time")
10 ax.set_ylabel(r"$x$")
11 ax.legend()
12
13 fig.colorbar(im, ax=ax, label=r"$u(t,x)$")
```

If the line does not follow the visible structure, the feature extraction method needs to be inspected.

→ **Example 02 - 04**

Vector fields with quiver

Some fields have several components at each point. A two-dimensional vector field can be inspected with arrows, but dense arrows quickly become unreadable.

```
1 xv = np.linspace(-2.0, 2.0, 25)
2 yv = np.linspace(-2.0, 2.0, 25)
3 Xv, Yv = np.meshgrid(xv, yv, indexing="ij")
4
5 Vx = -Yv
6 Vy = Xv
7
8 fig, ax = plt.subplots(figsize=(4.5, 4.0))
9 ax.quiver(Xv, Yv, Vx, Vy)
10
11 ax.set_xlabel(r"$x$")
12 ax.set_ylabel(r"$y$")
13 ax.set_aspect("equal")
```

This plot is mainly qualitative unless the arrow scale is calibrated and explained.

Field comparisons need shared colour limits

When comparing two fields, using separate automatic colour scales can hide the actual difference. Shared limits make the comparison honest.

```
1 rho_a = rho2
2 rho_b = 0.8 * np.exp(-((X2 - 1.2)**2/(2*sigma_x**2)
3               + (Y2 + 0.2)**2/(2*sigma_y**2)))
4
5 vmin = min(rho_a.min(), rho_b.min())
6 vmax = max(rho_a.max(), rho_b.max())
7
8 fig, axes = plt.subplots(1, 2, figsize=(8.0, 3.2), sharex=True, sharey=True)
9 for ax, field, title in zip(axes, [rho_a, rho_b], ["case A", "case B"]):
10     pcm = ax.pcolormesh(X2, Y2, field, shading="auto", vmin=vmin, vmax=vmax)
11     ax.set_title(title)
12     ax.set_aspect("equal")
13
14 fig.colorbar(pcm, ax=axes, label=r"$\rho$")
```

Difference plots complete the comparison

Two fields may look similar while their difference has systematic structure. A direct difference plot often reveals shifts, phase errors, boundary artefacts, or amplitude errors.

```
1 D = rho_b - rho_a
2 limit = np.max(np.abs(D))
3
4 fig, ax = plt.subplots(figsize=(5.5, 3.5))
5 pcm = ax.pcolormesh(X2, Y2, D, shading="auto", cmap="RdBu_r",
6                     vmin=-limit, vmax=limit)
7
8 ax.set_xlabel(r"$x$")
9 ax.set_ylabel(r"$y$")
10 ax.set_aspect("equal")
11 ax.set_title(fr"difference,  $|D| \rightarrow \infty = \{limit:.2e\}$ ")
12 fig.colorbar(pcm, ax=ax, label=r"$\rho_B - \rho_A$")
```

For numerical validation, a difference plot is often more valuable than two separate field plots.

Saving figures reproducibly

A figure that may be used in a report, notebook, or article should be saved explicitly. The filename, resolution, bounding box, and format are part of the workflow.

```
1 fig, ax = plt.subplots(figsize=(5.5, 3.5))
2 pcm = ax.pcolormesh(X2, Y2, rho2, shading="auto")
3 fig.colorbar(pcm, ax=ax, label=r"$\rho$")
4
5 ax.set_xlabel(r"$x$")
6 ax.set_ylabel(r"$y$")
7 ax.set_aspect("equal")
8
9 fig.savefig("field_diagnostic.png", dpi=200, bbox_inches="tight")
10 fig.savefig("field_diagnostic.pdf", bbox_inches="tight")
```

Raster formats are convenient for images and slides. Vector formats are usually better for line plots and publication graphics.

A compact figure function

For repeated diagnostics, it is useful to wrap plotting conventions into a small function. This reduces accidental differences between figures.

```
1 def plot_field(x, y, field, label=r"$u$", title=None):
2     X, Y = np.meshgrid(x, y, indexing="ij")
3
4     fig, ax = plt.subplots(figsize=(5.5, 3.5))
5     pcm = ax.pcolormesh(X, Y, field, shading="auto")
6     fig.colorbar(pcm, ax=ax, label=label)
7
8     ax.set_xlabel(r"$x$")
9     ax.set_ylabel(r"$y$")
10    ax.set_aspect("equal")
11    if title is not None:
12        ax.set_title(title)
13
14    return fig, ax
```

This is not a plotting library. It is a way to keep diagnostics consistent across a notebook or project.

A final visual sanity checklist

Before trusting a visual numerical experiment, check both the array and the plot.

```
1 print("field shape:", rho2.shape)
2 print("x range:", x2.min(), x2.max(), "nx =", x2.size)
3 print("y range:", y2.min(), y2.max(), "ny =", y2.size)
4 print("finite:", np.isfinite(rho2).all())
5 print("range:", np.nanmin(rho2), np.nanmax(rho2))
```

Useful final questions:

- ▶ Do array axes match plotted axes?
- ▶ Are coordinates, units, and colour scales explicit?
- ▶ Is the aspect ratio physically meaningful?
- ▶ Are residuals, differences, or convergence checks shown when needed?
- ▶ Does the figure agree with simple numerical diagnostics?

What this lecture should leave behind

The main message is that NumPy and Matplotlib are not separate topics. In scientific computing, arrays and figures form one diagnostic workflow.

construct → inspect → transform → diagnose → visualize

A good numerical plot is not only attractive. It is traceable to arrays, coordinates, assumptions, and checks.

The same habits will be useful when we move to data pipelines, optimization, machine-learning-oriented workflows, and performance-aware code.

Section 4

Further reading

Supplementary literature

- ▶ Harris, C.R., Millman, K.J., van der Walt, S.J. *et al.* Array programming with NumPy. *Nature* **585**, 357–362 (2020). DOI: <https://doi.org/10.1038/s41586-020-2649-2>
- ▶ Hunter, J.D. Matplotlib: A 2D graphics environment. *Computing in Science & Engineering* **9**(3), 90–95 (2007). DOI: <https://doi.org/10.1109/MCSE.2007.55>
- ▶ Rougier, N.P., Droettboom, M., Bourne, P.E. Ten Simple Rules for Better Figures. *PLOS Computational Biology* **10**(9), e1003833 (2014). DOI: <https://doi.org/10.1371/journal.pcbi.1003833>
- ▶ Crameri, F., Shephard, G.E., Heron, P.J. The misuse of colour in science communication. *Nature Communications* **11**, 5444 (2020). DOI: <https://doi.org/10.1038/s41467-020-19160-7>